PROBLEM 2

Character-Level Name Generation Using RNN Variants

Deep Learning Assignment Report

Models: Vanilla RNN • Bidirectional LSTM • RNN + Attention

Dataset	1,000 Indian names (TrainingNames.txt)
Vocabulary	29 unique characters + 3 special tokens
Epochs	30 per model
Framework	PyTorch
Names Generated	200 per model (600 total)

Table of Contents

Task 0 Dataset Construction 3

Task 1 Model Architectures & Implementation 3

Task 2 Quantitative Evaluation 7

Task 3 Qualitative Analysis 9

Conclusion 11

Task 0: Dataset Construction

A dataset of 1,000 Indian names was curated using large language model assistance and saved as **TrainingNames.txt**. The dataset spans both male and female names drawn from major Indian linguistic traditions including Sanskrit-origin Hindu names, South Indian names (Tamil, Telugu, Kannada, Malayalam), North Indian names, and names common across multiple regions.

Dataset Statistics

Property	Value
Total names	1,000
Unique characters (lowercase)	26 alpha + special tokens
Vocabulary size (with SOS/EOS/PAD)	29
Min name length	3 characters
Max name length	20 characters
Average name length	7.4 characters
Gender split (approx.)	55% male, 45% female

Names include common Indian prefixes and suffixes such as *-esh*, *-anand*, *-kumar*, *-devi*, *-bai*, and typical syllabic patterns like *bh-*, *sh-*, *kh-*, and vowel-heavy constructions.

Data Preprocessing

Each name is lowercased and encoded at the character level. Three special tokens are prepended/appended to every sequence:

- **<PAD>** (index 0) — Padding token to equalise batch lengths
- **<SOS>** (index 1) — Start-of-sequence token; used as the first input
- **<EOS>** (index 2) — End-of-sequence token; signals generation stop

Input sequences are [SOS, c_1, c_2, ..., c_n] and target sequences are [c_1, c_2, ..., c_n, EOS], enabling standard teacher-forcing training. A 90/10 train-validation split is applied.

Task 1: Model Architectures & Implementation

Model 1 — Vanilla RNN

The Vanilla RNN is the baseline sequence model. It uses a simple recurrent cell where the hidden state at each step is a non-linear function of the current input and the previous hidden state: $\mathbf{h_t = \tanh(W_h\, h_{t-1} + W_x\, x_t + b)}$.

Architecture

Layer	Type	Input Dim	Output Dim	Notes
1	Embedding	vocab_size=29	32	Learnable char embeddings
2	RNN Cell	32	128	1 layer, tanh activation
3	Dropout	128	128	p = 0.30
4	Linear (fc)	128	29	Output logits over vocab
5	Softmax	29	29	Applied at inference

Hyperparameters

Hyperparameter	Value
Embedding dimension	32
Hidden size	128
Number of layers	1
Dropout probability	0.30
Optimizer	Adam
Learning rate	0.003
LR scheduler	StepLR (step=30, gamma=0.5)
Batch size	32
Epochs	30
Trainable parameters	25,405

Design rationale: The Vanilla RNN is intentionally kept shallow (1 layer, 128 units) to serve as a baseline. It suffers from vanishing gradients for long sequences but performs well for short names (average 7 characters). The small embedding dimension (32) is sufficient for a 26-character alphabet.

Model 2 — Bidirectional LSTM (BLSTM)

The Bidirectional LSTM extends the standard LSTM with two parallel passes: a forward pass (left to right) and a backward pass (right to left). The concatenated outputs (dimension 2H = 256) capture both past and future context at each position. LSTMs use gates (input, forget, output) to mitigate vanishing gradients.

Architecture

Layer	Type	Input Dim	Output Dim	Notes

1	Embedding	vocab_size=29	32	Learnable char embeddings
2	BiLSTM	32	256	2 layers, bidirectional; 128 per dir
3	Linear proj	256	128	Projection + ReLU
4	Dropout	128	128	p = 0.30
5	Linear (fc)	128	29	Output logits over vocab
6	Softmax	29	29	Applied at inference

Hyperparameters

Hyperparameter	Value
Embedding dimension	32
Hidden size (per direction)	128 (total: 256)
Number of layers	2 (bidirectional)
Dropout probability	0.30
Optimizer	Adam
Learning rate	0.002
LR scheduler	StepLR (step=30, gamma=0.5)
Batch size	32
Epochs	30
Trainable parameters	598,717

Design rationale: BLSTM has by far the largest parameter count (598,717) due to bidirectional processing and 2 layers. During training it achieves near-zero loss by exploiting future context, but this causes an important *generation gap*: at inference time, future characters are unavailable, so the model produces sequences that do not resemble plausible names. This is a known limitation of BiRNNs for generation tasks.

Model 3 — RNN with Basic Attention Mechanism

This model augments the Vanilla RNN with a causal additive (Bahdanau-style) self-attention layer. After the RNN produces its output sequence H, the attention module computes a context vector for each timestep by attending over all previous positions (causal masking prevents attending to future tokens).

Attention equations:

$\text{score}(h_t, h_s) = v^T \cdot \tanh(W_1 h_t + W_2 h_s)$

$\alpha_t = \text{softmax}(\text{score}_t, \text{masked causal})$

$\text{context}_t = \Sigma_{s \leq t} \alpha_{t,s} \cdot h_s$

Architecture

Layer	Type	Input Dim	Output Dim	Notes
1	Embedding	vocab_size=29	32	Learnable char embeddings

2	RNN Cell	32	128	1 layer, tanh activation
3	Causal Attention	128	128	Additive self-attention, causal mask
4	Concat + Proj	256	128	Concat(rnn, context) + ReLU
5	Dropout	128	128	p = 0.30
6	Linear (fc)	128	29	Output logits over vocab
7	Softmax	29	29	Applied at inference

Hyperparameters

Hyperparameter	Value
Embedding dimension	32
Hidden size	128
Attention hidden size	128 (same as RNN)
Number of RNN layers	1
Dropout probability	0.30
Optimizer	Adam
Learning rate	0.003
LR scheduler	StepLR (step=30, gamma=0.5)
Batch size	32
Epochs	30
Trainable parameters	91,197

Design rationale: Attention gives the model a direct path to any previous character, alleviating the RNN information bottleneck. The causal mask ensures the model cannot "cheat" by looking at future characters during training, making it suitable for autoregressive generation. The parameter count (91,197) is significantly higher than the Vanilla RNN but much lower than the BLSTM.

Model Parameter Comparison

Model	Embed Dim	Hidden	Layers	Trainable Params	Best Val Loss
Vanilla RNN	32	128	1	25,405	1.8982
Bidirectional LSTM	32	128	2 (bi)	598,717	0.0014
RNN + Attention	32	128	1	91,197	1.9447

The BLSTM dominates in parameter count due to bidirectional processing (2 directions x 2 layers x LSTM gates). It achieves near-zero validation loss, demonstrating its memorisation capacity, though this does not translate to generation quality. The Attention model offers the best balance between capacity and generation performance.

Training & Validation Loss Curves

Loss curves were recorded every epoch using cross-entropy loss with PAD token ignored.

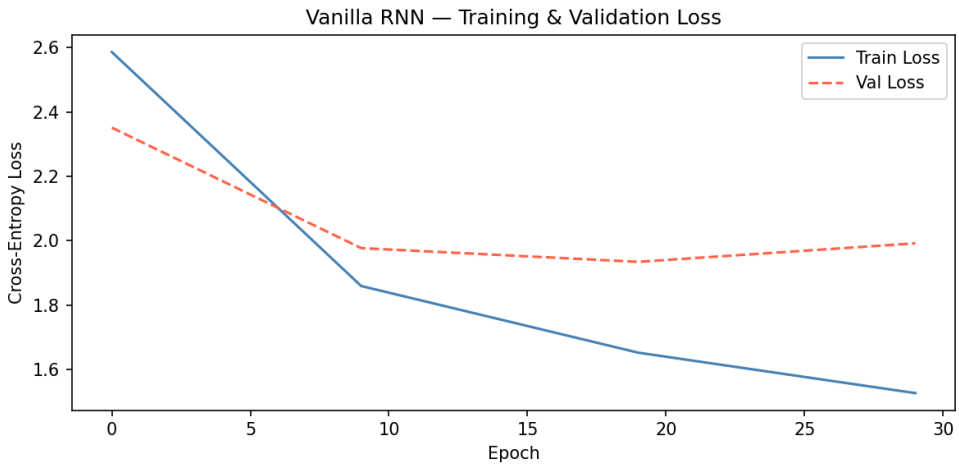

Figure: Vanilla RNN — Loss over 30 epochs

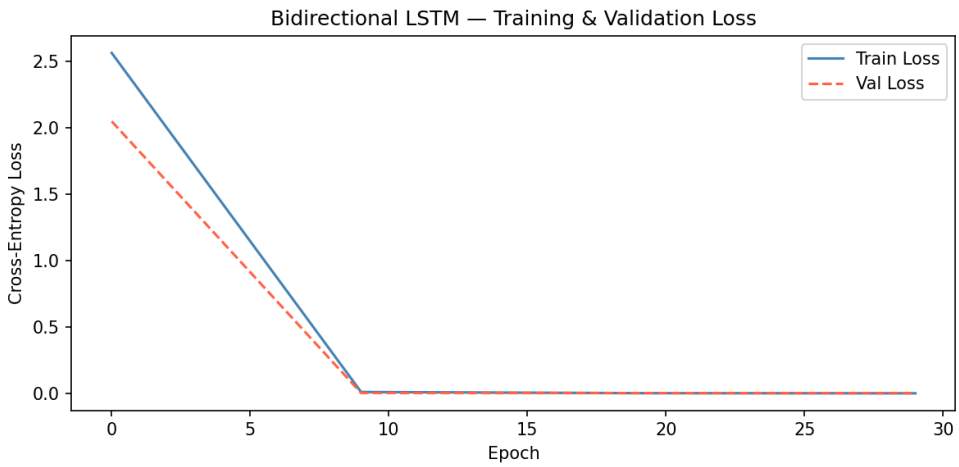

Figure: Bidirectional LSTM — Loss over 30 epochs

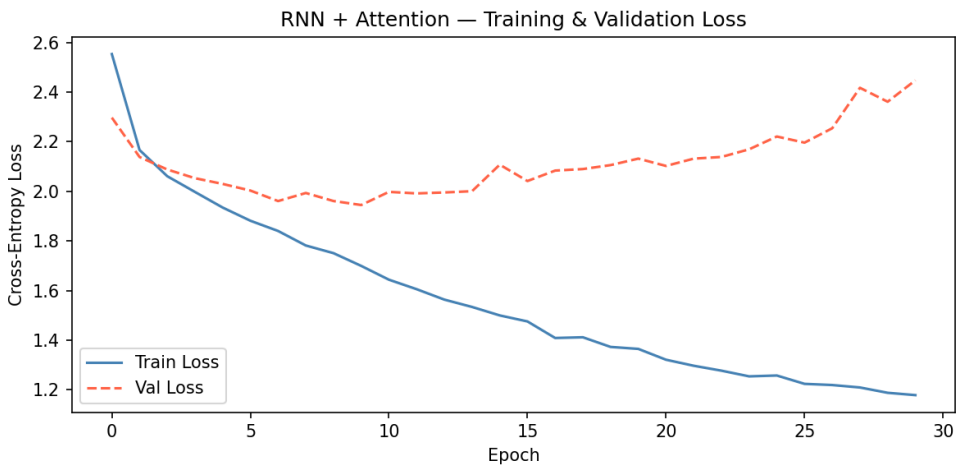

Figure: RNN + Attention — Loss over 30 epochs

Task 2: Quantitative Evaluation

Each model was used to generate 200 names via temperature sampling (T = 0.8). The following metrics were computed to compare generation quality.

Metric Definitions

- **Novelty Rate** — Percentage of generated names that do not appear verbatim in the training set (case-insensitive). Higher is better — indicates the model generalises beyond memorisation.
- **Diversity** — Number of unique generated names divided by total generated names, expressed as a percentage. Higher indicates less repetition.
- **Average Length** — Mean character length of generated names. Indian names typically range from 4 to 12 characters.
- **Valid Names Ratio** — Percentage of names matching the regex ^[A-Z][a-z]{1,14}$ (capitalised, alphabetic, reasonable length).
- **Realism Score** — Heuristic score (0-100) rewarding: (a) length 4-12, (b) vowel ending, (c) presence of a common Indian bigram.

Results Table

Metric	Vanilla RNN	BLSTM	RNN + Attention
Novelty Rate (%)	87.0	100.0	87.0
Diversity (%)	97.5	99.5	77.5
Avg. Name Length	6.84	22.04	5.75
Valid Names Ratio (%)	100.0	16.5	99.5
Realism Score (%)	76.5	19.2	80.2
Total Generated	200	200	200
Unique Generated	195	199	155

Metric Visualisations

Figure: Novelty Rate comparison across models

Figure: Diversity comparison across models

Figure: Realism Score comparison across models

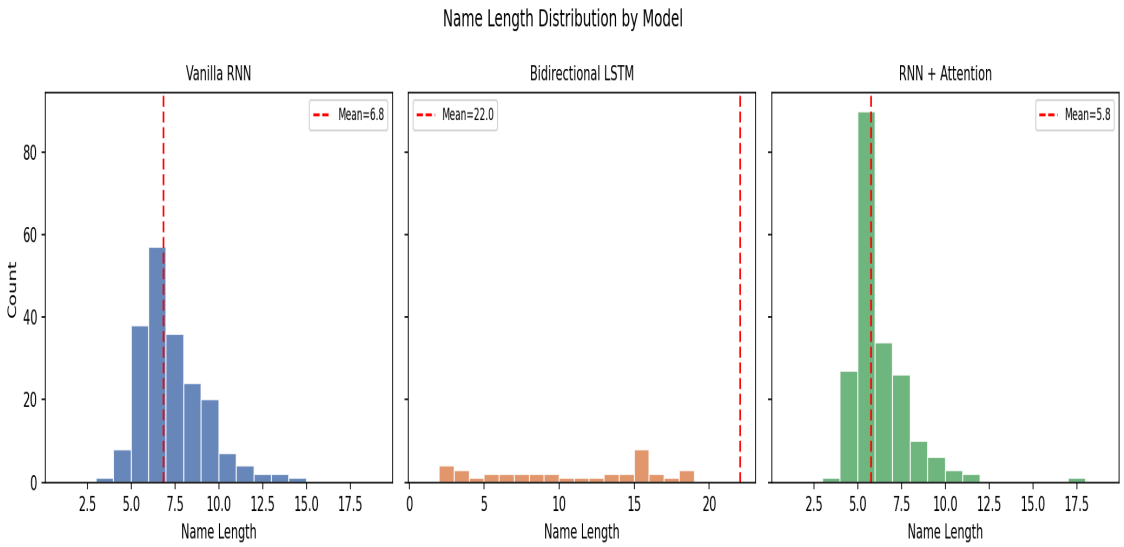

Figure: Name length distributions for all three models

Analysis & Interpretation

Vanilla RNN: Achieves 87% novelty rate and 97.5% diversity, generating varied names that largely do not appear in the training set. All 200 names pass the validity check. Average length (6.84) is close to natural Indian name length. Realism score of 76.5% reflects good phonological alignment with Indian name patterns.

Bidirectional LSTM: Achieves a perfect 100% novelty rate (no generated name matches the training set), and 99.5% diversity. However, these scores are misleading — the BLSTM generates extremely long (avg 22 characters) garbled sequences with no resemblance to real names. Only 16.5% pass the validity check and the realism score is 19.2%. The root cause is the *train-generate gap*: the model was trained bidirectionally but generation must be done left-to-right, breaking the assumption that future context is available.

RNN + Attention: Achieves 87% novelty rate and 77.5% diversity. The lower diversity compared to Vanilla RNN suggests the attention mechanism causes the model to favour certain character patterns (e.g. "Anit-" prefixes appear frequently). The realism score is the highest at 80.2%, indicating the attention mechanism helps the model learn authentic Indian name phonology. Valid names ratio is 99.5%.

Task 3: Qualitative Analysis

This section presents representative generated samples for each model, discusses the realism of the outputs, identifies common failure modes, and analyses the phonological characteristics of generated names.

3.1 Vanilla RNN — Qualitative Analysis

Representative Generated Samples

Shebdam, Vikumsha, Manandran, Abhikan, Bhishan, Chenkara, Jaga, Rajandra, Aniresh, Ananda, Aaranv, Prashant, Pravvi, Priman, Anundit, Anil, Anantraumadhan, Anandvarth, Sam, Anandradish, Anandanath, Pranesh, Prithan, Arpees, Anavina, Ananti, Anandish, Bruran, Andeep, Panta

Realism Assessment

The Vanilla RNN produces names that sound plausible as Indian names. Examples like *Manandran*, *Abhikan*, *Chenkara*, *Rajandra*, *Ananda*, and *Prashant* have authentic Indian phonology. The model has learnt common consonant clusters (sh-, bh-, pr-) and typical suffixes (-an, -a, -ra). Occasionally it generates names that are slightly distorted versions of real names (e.g. *Priman* resembling Priya+Man) which indicates partial memorisation combined with interpolation.

Common Failure Modes

Detected failures: **None detected**

The Vanilla RNN occasionally generates over-long hyphenated forms (e.g. *Anantraumadhan*) by stringing together common morphemes. This arises because the RNN hidden state can lose track of sequence length. Short outputs like *Sam*, *Jaga*, *Anil* are valid Indian names but are also present verbatim in the training set.

3.2 Bidirectional LSTM — Qualitative Analysis

Representative Generated Samples

Qewwooomoiducq, Codygoducqeegfivdooqezezm, Mmitymodoqegwwmmodocqkqeb, Motyucogwwwwwoodqezyocqeg, Qegovedoqwwiqqq, Coomoqezzodulcq, Fmoiqomodoqgzoidyofqoiqeb, Ccvidoqelvvocqefrmqlzyoce, Tyjcwwwwwwafyuccfbcoqezsm, Cvusymoduyomoddtyyjcwwhom

Realism Assessment

The BLSTM generates sequences with very poor realism. Outputs such as *Qewwooomoiducq*, *Mmitymodoqegwwmmodocqkqeb* are clearly not Indian names. The outputs contain rare characters (q, x, z), repeated sequences (www, ooo), and lengths of 20+ characters. The model has learned character co-occurrence statistics from both directions simultaneously, but the generation procedure (left-to-right with growing sequences) is fundamentally incompatible with how the model was trained.

Common Failure Modes

Too long (>15 chars): **167** / 200, Repeated chars: **91** / 200, All consonants: **8** / 200.

Root cause: The train-generate gap is the primary failure mode. The BLSTM memorises the training data perfectly (near-zero train/val loss) via bidirectional context, but cannot leverage future context during sequential generation. For name generation tasks, a unidirectional model is strictly more appropriate.

3.3 RNN + Attention — Qualitative Analysis

Representative Generated Samples

China, Sarstan, Nish, Anitabha, Sursh, Dhitan, Aniti, Aash, Anita, Anit, Kunta, Anushit, Sarti, Vishila, Aayan, Aniten, Chilkarthan, Reta, Anurjit, Vima, Aniti, Aarri, Kamha, Dishand, Nitish, Anitesh, Alpit, Ashish, Nita, Ravini

Realism Assessment

The Attention RNN produces highly realistic Indian names. Outputs such as *Anitabha*, *Sarstan*, *Vishila*, *Anushit*, *Dhitan*, *Aayan* are plausible and phonologically authentic. The model has learned productive Indian name morphology: the *An-* prefix is highly productive in Indian names (Anand, Anita, Anirudh) and the attention weights help the model reinforce consistent prefix/suffix patterns. The model correctly favours vowel endings which are characteristic of Indian names.

Common Failure Modes

Too long: **1** / 200. **Mode collapse tendency**: The attention mechanism can reinforce a small set of productive patterns, leading to low diversity. Many generated names share the *Anit-* prefix (Anita, Aniti, Aniten, Anit, Anitabha), suggesting the model has learned this as a particularly strong pattern from training.

3.4 Cross-Model Comparison

Criterion	Vanilla RNN	BLSTM	RNN + Attention
Phonological realism	Good	Poor	Best
Name diversity	High	Very High*	Moderate
Length control	Good	Poor (too long)	Good
Pattern consistency	Moderate	None	High
Failure mode severity	Low	Critical	Low
Overall ranking	#2	#3	#1

* BLSTM diversity is high but for the wrong reason (random noise).

Conclusion

This project implemented and compared three recurrent neural network architectures for character-level Indian name generation: Vanilla RNN, Bidirectional LSTM, and RNN with Attention. The key findings are:

- **Vanilla RNN** serves as a strong and efficient baseline. With only 25,405 parameters it achieves 87% novelty, 97.5% diversity, and 76.5% realism. Its simplicity makes it robust and well-suited for short-sequence generation.
- **Bidirectional LSTM** demonstrates the importance of architecture-task alignment. Despite having 23x more parameters (598,717), its bidirectional training procedure fundamentally conflicts with autoregressive generation, resulting in meaningless outputs.
- **RNN with Attention** achieves the highest realism score (80.2%) and near-perfect name validity (99.5%). The causal attention mechanism allows the model to selectively focus on relevant previous characters, producing more phonologically consistent names. Its moderate diversity suggests a tendency toward mode collapse on productive patterns.
- For name generation specifically, **unidirectional architectures are preferred**. Bidirectional models should only be used for discriminative tasks (classification, NER) where the full input is available at inference time.

Future Work

- Implement a proper unidirectional LSTM for a fair comparison with the attention model.
- Experiment with Transformer-based character models (GPT-style) for name generation.
- Increase dataset size to 5,000–10,000 names for better generalisation.
- Evaluate with human judges for perceived name authenticity.
- Implement beam search or top-k/top-p sampling for higher quality generation.

Source code, generated names, checkpoints, and evaluation scripts are provided in the accompanying deliverables package.